

arguments that the programmer specifies while writing the code, it's better to use the `*` syntax to accept a varying number of arguments.

Using ** to Create Variadic Functions

Variadic functions can use the `**` syntax, too. Although the `*` syntax in `def` statements represents a varying number of positional arguments, the `**` syntax represents a varying number of optional keyword arguments.

If you define a function that could take numerous optional keyword arguments without using the `**` syntax, your `def` statement could become unwieldy. Consider a hypothetical `formMolecule()` function, which has parameters for all 118 known elements:

```
>>> def formMolecule(hydrogen, helium, lithium, beryllium, boron, --snip--
```

Passing 2 for the hydrogen parameter and 1 for the oxygen parameter to return 'water' would also be burdensome and unreadable, because you'd have to set all of the irrelevant elements to zero:

```
>>> formMolecule(2, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 --snip--
'water'
```

You could make the function more manageable by using named keyword parameters that each have a default argument, freeing you from having to pass that parameter an argument in a function call.

NOTE

Although the terms *argument* and *parameter* are well defined, programmers tend to use *keyword argument* and *keyword parameter* interchangeably.

For example, this `def` statement has default arguments of `0` for each of the keyword parameters:

```
>>> def formMolecule(hydrogen=0, helium=0, lithium=0, beryllium=0, --snip--
```

This makes calling `formMolecule()` easier, because you only need to specify arguments for parameters that have a different value than the default